Week # 01

Date _____

Introduction To Requirement Engineering

1) Introduction :-

Software development begins by understanding what the system should do and what the users expect from it. This understanding is captured through requirements.

Requirements engineering is the process of discovering, analyzing, documenting, and managing the needs and expectations of stakeholders for a software system. It plays a critical role in the success of software projects because incorrect or incomplete requirements can lead to system failure.

The main goal of requirements engineering is to ensure that the system being developed satisfies the needs of its users and stakeholders.

2) What are Software Requirements?

Software requirements describe the services that a system should provide and the constraints under which the system must operate. These requirements reflects the needs of the customers and the purpose of the system.

Scanned with CamScanner

Date

In simple terms:

"Requirements explain what a system should do and how should it behave."

* Requirements are important because they act as a bridge between the customer and the development team.

3) Importance of Requirement Engineering :-

Requirements engineering is essential for several reasons:

- It helps developers understand what the customer actually wants.
- It reduces misunderstandings between the developers and clients.
- It ensures that the final software system satisfies user needs.
- It helps avoid costly mistakes during later stages of development.
- It provides a clear foundation for system design and development.

If requirements are not properly defined, the software system may fail to meet user expectations.

Scanned with CamScanner

Date

4) Stakeholders in Requirements Engineering :-

Stakeholders are individuals or groups who have an interest in the system or are affected by it.

Examples of stakeholders include:

- End Users
- Customers or clients
- System developers
- Project managers
- System testers
- Maintenance engineers

Each stakeholder may have different expectations from the system, so it is important to gather requirements from all relevant stakeholders.

5) Types of Software Requirements :-

Software requirements are generally classified into two main categories:

1) Functional Requirements.
2) Non-Functional Requirements.

6) Functional Requirements :-

It describe the services or functions that the system must perform.

Date

They specify what the system should do in response to specific inputs or situations.

- Examples:

- The system shall allow users to create accounts.
- The system shall allow wers to search for products.
- The system shall generate daily reports.

* Functional requirements focus on the behavior of the system.

7) Non-Functional Requirements :-

It describe the constraints and quality attributes of the system. They define how well the system should perform rather than what the system should do.

- Examples:

- The system should respond within 2 seconds.
- The system should be secure and protect user data.
- The system should be available 24/7.

* Non-functional requirements often apply to the entire system rather than individual features.

Scanned with CamScanner

Date

8) Domain Requirements :-

Domain requirements come from the application domain of the system, not directly from the users. These requirements depend on the specific field where the system will be used.

- Examples:

- Medical systems must follow healthcare regulations.
- Banking systems must follow financial security rules.

* Domain requirements may introduce new functional requirements or place constraints on existing ones.

9) User Reqr Vs System Reqr :-

Requirements can also be categorized based on their level of detail.

User Requirements :-

User requirements describe the services the system should provide in a simple and understandable form. They are usually written in natural language & diagrams.

Example: The system should allow users to upload photos.

Scanned with CamScanner

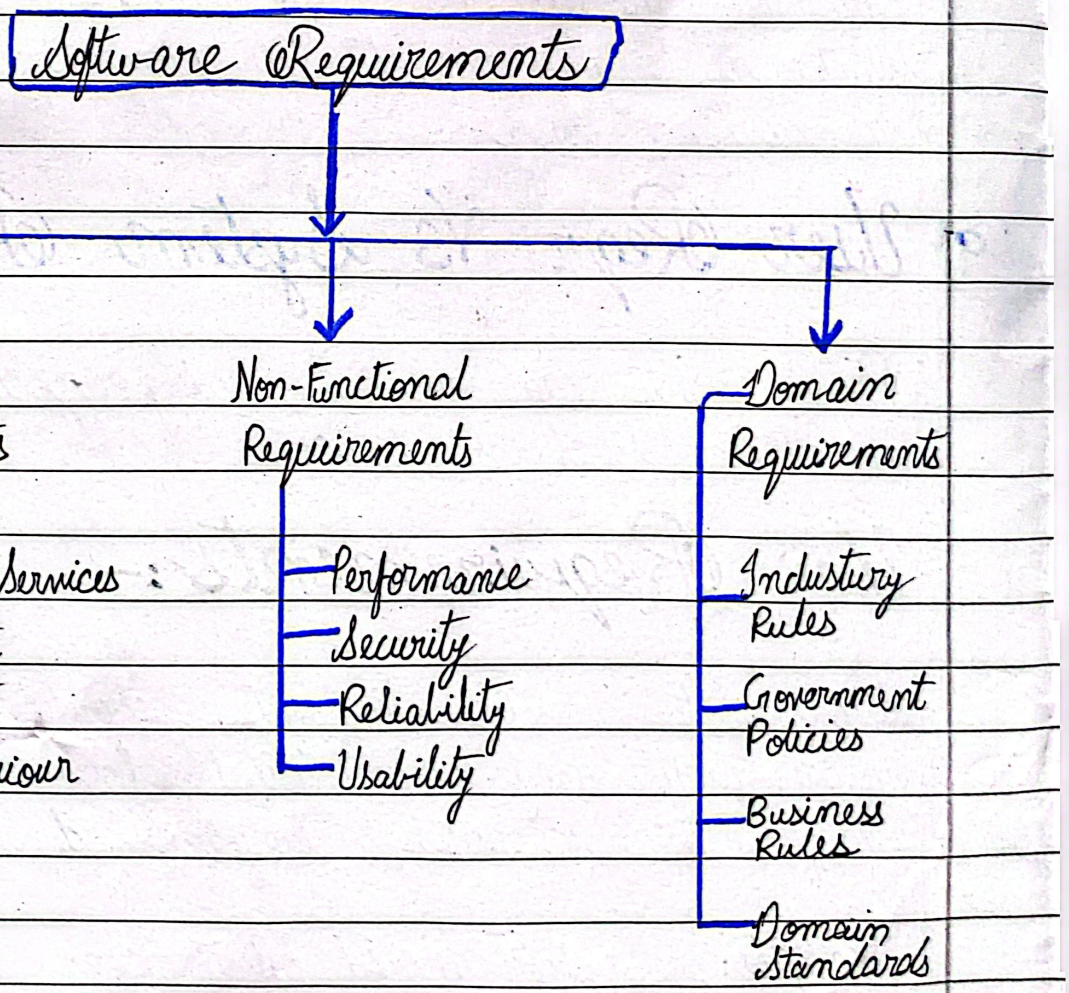
Date

System Requirements :-

System Requirements provide a detailed description of system function and constraints. They are mainly used by developers and engineers.

Example: The system shall allow users to upload images in JPG or PNG format with a maximum size of 5 MB.

10) Diagram :-



Date

11) Example Case Study :-

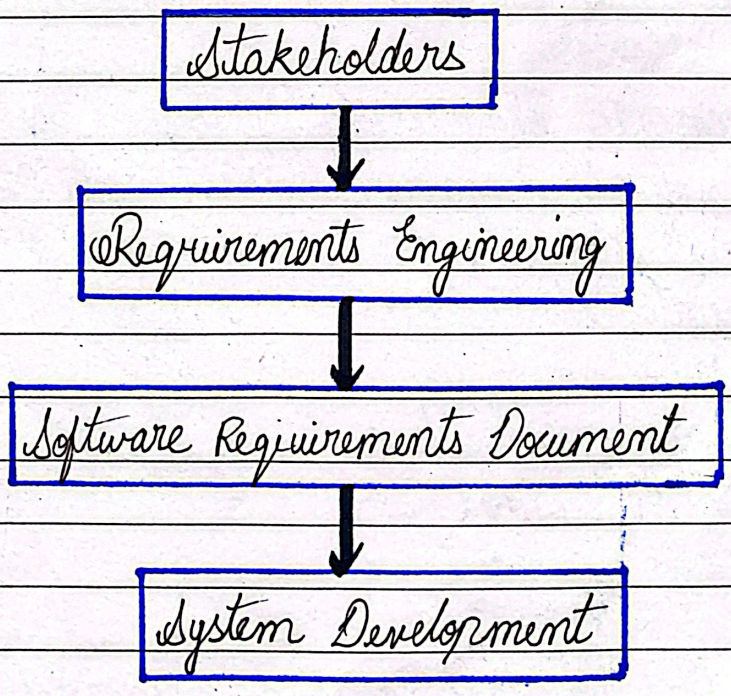
Example: Online Shopping System.

★ Functional Requirements:

- Users can register and login
- Users can search for products
- Users can place orders.

★ Non-Functional Requirements:

- The system should load pages within 3 seconds.
- The system must protect customer payments data.
- The system should support thousands of users simultaneously.



Process Diagram

Scanned with CamScanner

Week #02

08

Date

Requirements Process And Requirement Analysis :-

1) Requirement Engineering Process :-

The requirements engineering process is a structured set of activities used to discover, analyze, document, and validate the requirements of a software system.

This process helps developers understand what the system should do and ensures that the final product satisfies the needs of stakeholders.

The requirements engineering process generally includes several major activities such as:

- Requirements elicitation
- Requirements analysis
- Requirements specification
- Requirements validation
- Requirements management

These activities are not always performed in a strict sequence. In practice, they are iterative and interleaved, meaning that developers may

Date

move back and forth between activities as their understanding of the system improves.

The final output of this process is a document called the Software Requirement Specification (SRS) which describes the system requirements in detail.

2) Major Activities in the Requirements Process :-

2.1) Requirements Elicitation :-

It is the process of gathering information from stakeholders about the system they want to build.

During this stage, software engineers interact with users, managers, and other stakeholders to understand:

- The services the system should provide.
- The expected performance of the system.
- Constraints such as hardware, budget or regulations.

However eliciting requirements is often difficult because stakeholders may not clearly know what they want or may express their needs in vague terms.

Scanned with CamScanner

Date

2.2 Requirement Analysis :-

It involves examining and organizing the collected requirements to understand them clearly.

During analysis, engineers :

- Identify conflicts between requirements.
- Organize requirements into logical groups.
- Prioritize requirements based on importance.
- Resolve inconsistencies between stakeholders.

This stage ensures that the requirements are complete and understandable before documentation begins.

2.3 Requirements Specification :-

It is the process of documenting the requirements in a clear and structured way. The result of this stage is the (SRS) document, which acts as a formal agreement between the client and the development team.

The SRS document includes :

- Functional requirements.
- Non-functional requirements
- System constraints
- Diagrams and models.

Scanned with CamScanner

Date

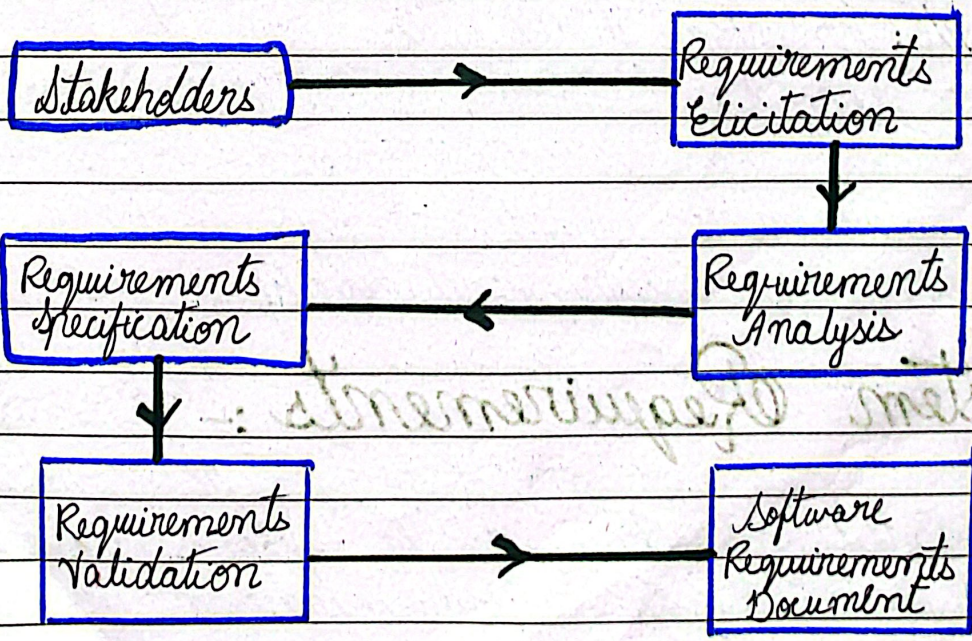
This document serves as the foundation for system design and development.

2.4 Requirements Validation :-

It ensures that the documented requirements correctly represent the needs of stakeholders.
During validation, engineers check the requirement for:

- Correctness
- Completeness
- Consistency
- Feasibility

Validation is important because errors in requirements can lead to expensive mistakes during development.



Date

3) Levels (Layers) of Requirements :-

Requirements are written at different levels of detail because different stakeholders need different types of information. The two main levels of requirements are :

1) User Requirements
2) System Requirements

3.1) User Requirements :-

It describe the system services in a simple and understandable way. They are usually written in natural language and diagrams so that non-technical stakeholders such as managers or clients can understand them.

Example: The system should allow user to upload photos.

User requirements provide a high level decription of system functionality.

3.2 System Requirements :-

It provide a detailed description of the system functions and constraints. These requirements are

Date

written for developers and engineers who will design and implement the system.

Example: The system shall allow users to upload image files in JPG or PNG format with maximum size of 5 MB.

System requirements expand the user requirements by adding technical details about system behavior.

4) Characteristics of Good Requirements:-

For requirements to be effective, they must possess certain important characteristics. A good requirement should have the following qualities:

1. **Correct**: A requirement must accurately describe the system functionality that stakeholders need.

2. **Complete**: A requirement specification should include all necessary system services & constraints. Incomplete requirements can cause system failures or misunderstandings during development.

3. **Consistent**: Requirements should not conflict with each other. For example, one requirement should not demand fast system response while another requires heavy processing that slows the system.

Scanned with CamScanner

Date

4. **Verifiable**: Each requirement should be measurable or testable so that developers can verify whether it has been implemented correctly.

5. **Feasible**: Requirements should be realistic and achievable within the available time, budget, and technology.

6. **Traceable**: Each requirement should be traceble to its source so that developers know why it exists and how it relates to other requirements. Traceability helps analyze the impact of requirement changes during development.

5) Analyzing Quality Requirements:-

Quality requirements are also known as non-functional requirements. These requirements define the quality attributes of a system, such as performance, reliability, and usability. Unlike functional requirements, quality requirements apply to the entire system rather than individual features.

Examples of Quality Requirements:-

Performance: The system should process atleast 100 transactions per second.

Scanned with CamScanner

Date

Reliability: The system should have 99.9% availability.

Usability: Users should be able to learn basic system operations within 30 minutes of training.

Security: Only authorized users should be able to access confidential data.

Maintainability: The system should be easy to update and modify without affecting other components.

6) Importance of Quality Requirements:-

Quality requirements are extremely important because they determine whether the system is practical and usable.

For Example:

- If performance requirements are not satisfied, the system may become to slow.
- If security requirements are ignored, sensitive information may be exposed.
- If reliability requirements are not met, the system may frequently fail.

In many cases, failing to meet non-functional requirements can make the entire system unusable.

Scanned with CamScanner

Date

7) Summary :-

The requirements engineering process is a structured approach used to identify, analyze, document and validate software requirements. It includes activities such as requirements elicitation, analysis, specification, and validation.

Requirements are written at different levels, including user requirements and system requirements, to meet the needs of different stakeholders. A good requirement must be correct, complete, consistent, verifiable, feasible and traceable.

Quality requirements play a vital role in ensuring that the system performs efficiently, securely, and reliably.

Scanned with CamScanner

Week # 03

Date _____

1) Software Requirements in the Context of Systems Engineering:-

In large projects, software is rarely developed alone. It is usually part of a larger system that includes hardware, software, people, procedure and data.

Systems engineering is the discipline that focuses on designing and managing these complex systems throughout their lifecycle.

In this context, software requirements must be considered together with other system components to ensure that the entire system works effectively.

A software system may interact with:

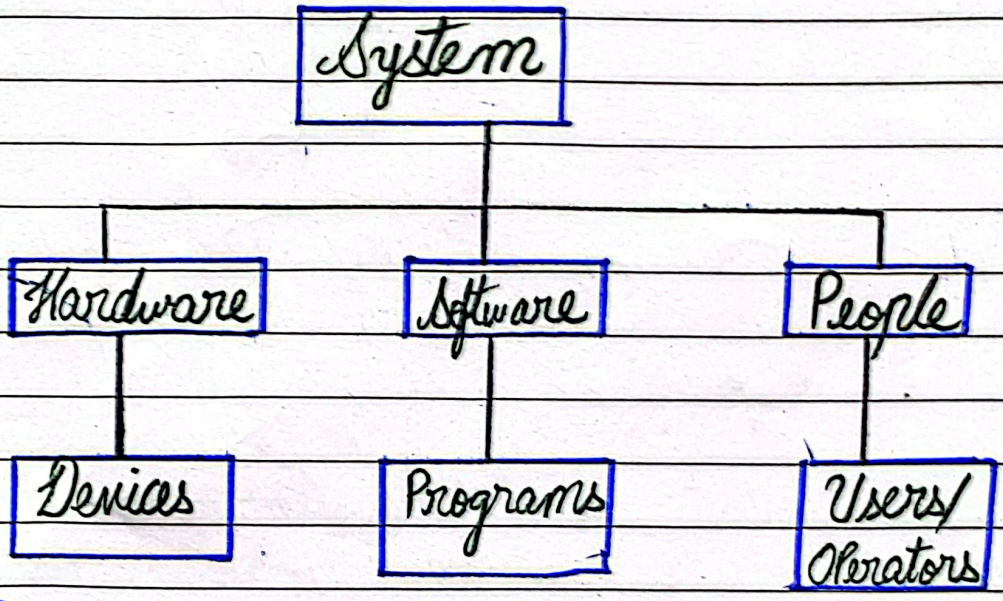
- hardware devices
- databases
- other software systems
- communication networks
- human users.

Therefore, requirements engineering must consider these interactions when defining system behavior.

Scanned with CamScanner

Date

The requirements of a system usually originate from stakeholders, who include users, managers, engineers, and other people who are affected by the system.



2) Relationship Between System And Software Requirement :-

System requirements describe the overall behavior of the ~~software~~ entire system, while software requirements describe the behavior of the software component within that system.

System requirements may include :

- ATM machine hardware

Scanned with CamScanner

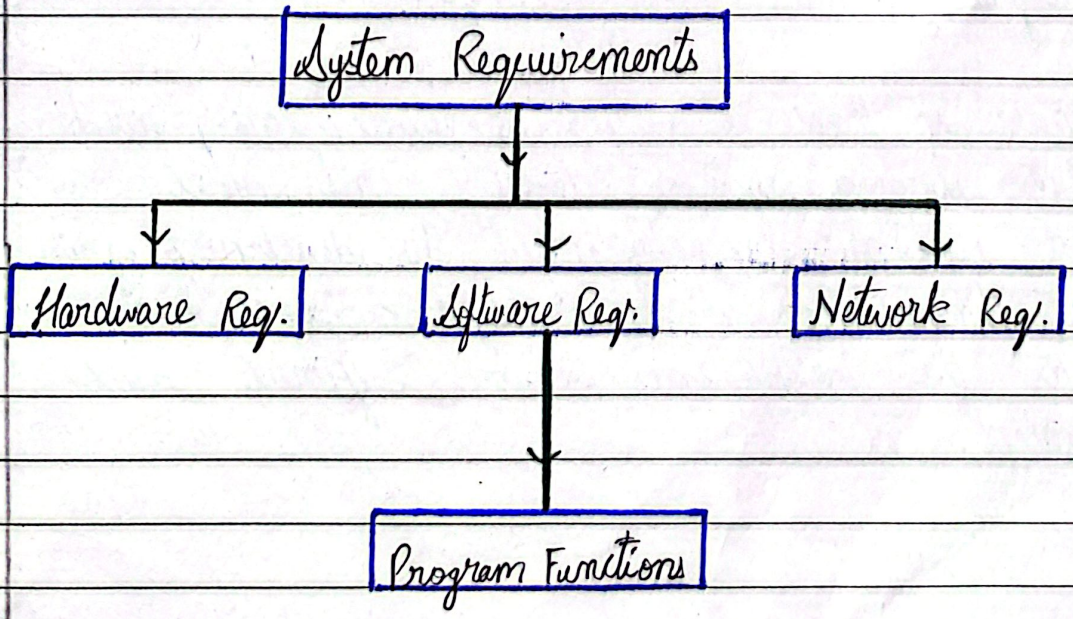
Date

- network connection
- cash dispenser device
- banking software

Software requirements specifically define:

- authentication of users
- transaction processing
- account balance checking
- money transfer function

Thus, software requirements are a subset of system requirements.



Date

3) Requirement Evolution :-

Requirements do not remain fixed throughout the life of a system. They change and evolve over time as organizations grow, technology improves, or users discover new needs.
The environment of a system is always changing due to:

- new business policies
- new laws or regulations.
- improvements in technology.
- feedback from users
- integration with new systems.

Because of these changes, requirements must evolve during system development and after system development. As developers gain more knowledge about the system and user needs, the requirements are refined and updated.

Scanned with CamScanner

Date

4) Types of Changing Requirements :-

Requirements can be categorized based on how frequently they change.

Enduring Requirements :-

Enduring requirements are stable requirements that rarely change. These requirements are usually related to the core activities of an organization.

Example:

- banks must record financial transactions.
- hospitals must store patient records.

These activities are fundamental and therefore remain stable for a long time.

Volatile Requirements :-

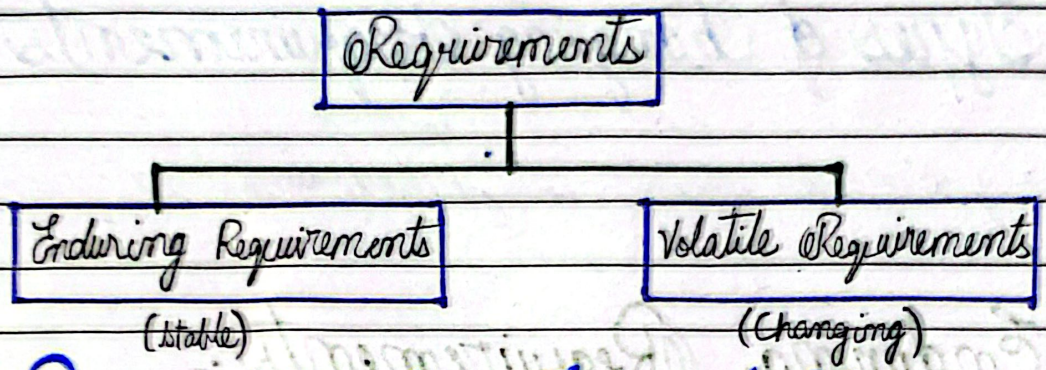
Volatile requirements are more likely to change frequently. These requirements are usually related to :

- User interface design
- reporting formats
- business rules

These aspects change as organizations update their processes or improve usability.

Scanned with CamScanner

Date



5) Requirements Traceability :-

Requirements traceability is the ability to track the relationships between requirements and other system elements. It allows engineers to understand :

- where a requirement originated.
- how it relates to other requirements
- how it effects system design and implementation.

Traceability is extremely important when requirements change because developers must understand how the change will effect other parts of the system.

Example: Requirement: "User must login before accessing the system".

This requirement may be connected to :

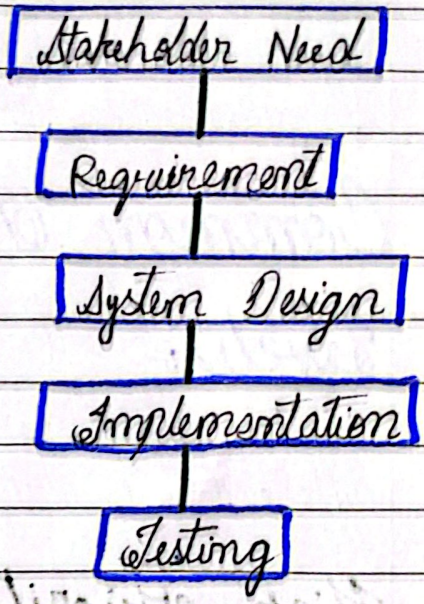
- authentication module

Date

- database user records
- security policies.

If this requirements changes, these components may also need modification.

Traceability ensures that every requirement can be tracked from its origin to its final implementation.



6) Requirements Prioritization :-

In most software projects, it is impossible to implement all requirements at the same time because of :

- limited time.
- limited budget.
- limited resources.

Therefore, requirements must be prioritized so that the most important features are developed first.

Date

Requirements prioritization helps development teams focus on delivering the highest value features early in the development process.

When multiple stakeholders are involved, their requirements may conflict. In such situations, negotiation is required to determine which requirements should have higher priority.

Common Requirement Priority Levels:-

Requirements are often categorized into different priority levels.

High Priority: Essential features that must be implemented for the system to function.

Example: User authentication in a banking system.

Medium Priority: Important features but not critical for the first release.

Example: Advanced search filters in an e-commerce system.

Low Priority: Features that are desirable but can be implemented later.

Scanned with CamScanner

Date

Example: Custom themes for the user interface.

7) Importance of Requirement Management:-

Since requirements change frequently, organizations must manage these changes effectively. Requirements management involves:

- identifying requirements
- documenting requirements
- tracking requirement changes
- controlling requirement updates

A formal change management process ensures that requirements changes are analyzed carefully before implementation.

8) Summary:-

Software requirements are often part of a larger system that includes hardware, people, and other components. Therefore, requirements engineering must consider the overall system context.

Requirements evolve over time due to changes in technology, business environments, and user needs. These changes make it necessary

Date

to manage and track requirements carefully.

Requirements traceability allows engineers to track relationships between requirements, design elements, and implementation components. Prioritization ensures that the most important system features are developed first.

Effective requirements management helps maintain system quality and ensures that changes are handled in a controlled and systematic manner.

Week # 04

1) Importance of Requirement Prioritization :-

Requirement prioritization is important for several reasons:

- it helps teams deliver critical features first.
- it ensures efficient use of development ~~featur~~ resources.
- it supports better planning & scheduling.
- it reduces project risks.

Scanned with CamScanner

Date

Without prioritization, development teams may waste time implementing less important features while critical features remain unfinished.

2) Factors Affecting Requirement Priority :-

Several factors influence how requirements are prioritized.

Business Value: Requirements that provide greater business benifits should be implemented earlier.

Example: An online store must prioritize the payment processing system before adding optional design features.

Implementation Cost: Some features require more development effort and cost. Engineers must evaluate whether the benefits justify the cost.

Risk Level: High-risk requirements should often be addressed early so that potential problems can be identified and solved.

Stakeholder Importance: Different stakeholders may have different priorities. Their influence within the organization can affect requirement prioritization.

Scanned with CamScanner

Date

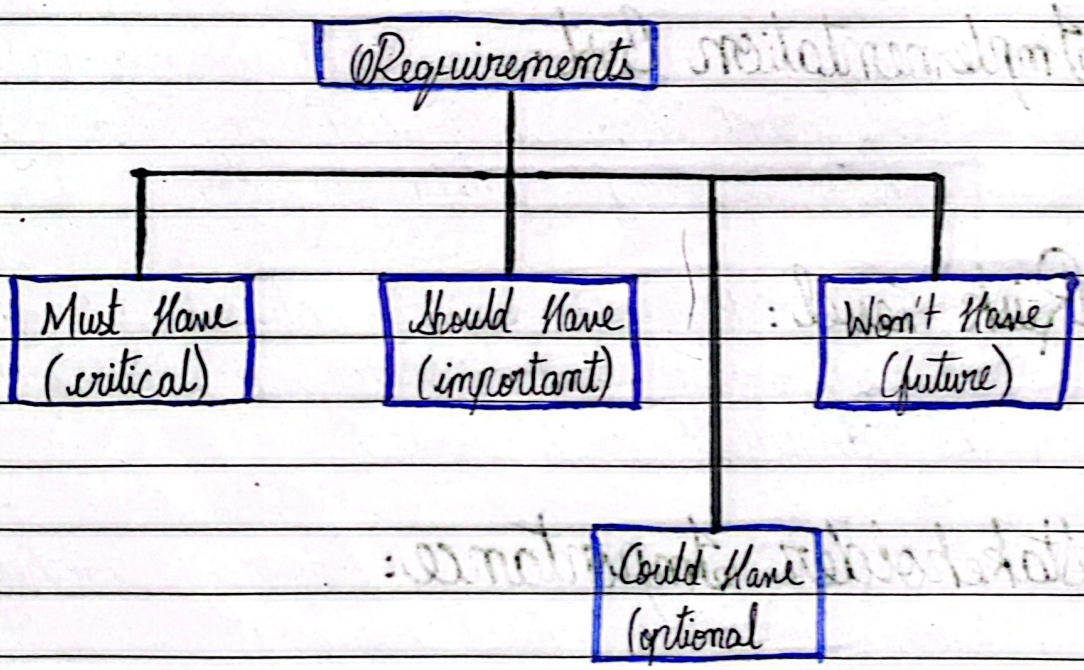
2) Techniques for Requirement Prioritization :-

Several techniques are used to prioritize requirements.

• Moscow Method :-

One common prioritization technique is the MoSCoW method. Requirements are divided into four categories :

- Must Have - essential requirements
- Should Have - important but not critical
- Could Have - desirable but optional
- Won't Have - not included in the current release.



Date

• Numerical Ranking :-

In this technique, each requirement is assigned a numerical value based on its importance.

Example:

Requirements	Priority
User login system	10
Product search	8
UI customization	4

Higher numbers indicate higher priority.

• Stakeholder Voting :-

In this method, stakeholders vote on the requirements they consider most important. Requirements receiving the highest number of votes are given higher priority.

4) Trade-off Analysis :-

Trade-off analysis is the process of evaluating conflicting requirements or design choices in order to determine the best solution.

Scanned with CamScanner

Date

In software systems, improving one quality attribute may negatively affect another.
For example:

- Increasing security may reduce system usability.
- improving performance may increase development cost.
- adding new features may increase system complexity.

Trade-off analysis helps engineers balance these conflicting factors to achieve the most effective system design.

5) Example of Trade-off Analysis :-

Consider a mobile banking application.
Two design options exist:

Option A: High security with multi-factor authentication.
Option B: Simple login process with faster access.
Option A provides better security but may reduce user convenience.
Option B improves usability but reduces security strength.

Engineers must analyze these trade-offs and determine which option best satisfies the system goals.

Scanned with CamScanner

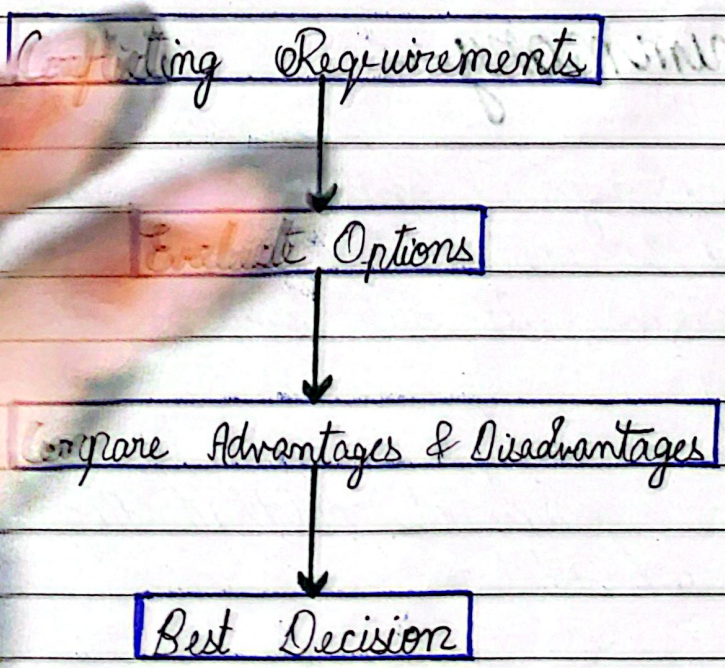
Date

6) Trade-off Analysis Process :-

The trade-off analysis process usually involves the following steps :

1. Identifying conflicting requirements.
2. Evaluate the impact of each option.
3. Analyze benefits and drawbacks.
4. Select the most balanced solution.

Trade-off Decision Process :-



Date

7) Benefits of Trade-off Analysis :-

Trade-off analysis provides several advantages:

- helps identify the best system design.
- improves decision-making.
- balances performance, cost & usability.
- reduces system design conflicts.

It ensures that the final system design satisfies the most important requirements while minimizing negative impacts.

8) Summary :-

Requirements prioritization helps software engineers determine which requirements should be implemented first based on their importance, cost, risk and business value.

Trade-off analysis is used to evaluate conflicting system requirements and identify the most balanced solution.

Both techniques are essential for effective software development and help ensure that projects are completed successfully within available resources.

Week # 05

1) Risk Analysis:

of identifying potential probl
negatively affect the succe
In software engineering, risk

- unclear requirements
- changing requirements
- technical complexity
- limited resources
- unrealistic deadlines

Risk analysis helps proje
problems early and prep

2) Types of Softwa

software projects can g
into three main cate

Project Risks: -

project schedule, budge

Examples:

- lack of skilled develop
- delays in project sc
- insufficient project

Week # 05

Date

1) Risk Analysis :-

Risk analysis is the process of identifying potential problems that could negatively affect the success of a software project. In software engineering, risks may arise due to :

- unclear requirements
- changing requirements
- technical complexity
- limited resources
- unrealistic deadlines

Risk analysis helps project teams anticipate problems early and prepare solution in advance.

2) Types of Software Risks :-

Risks in software projects can generally be classified into three main categories.

Project Risks : Project risks affect the project schedule, budget, or resources.

Examples:

- lack of skilled developers
- delays in project schedule
- insufficient project funding

Scanned with CamScanner

Technical Risks: Technical risks arise due to technical challenges or uncertainties.

Examples:

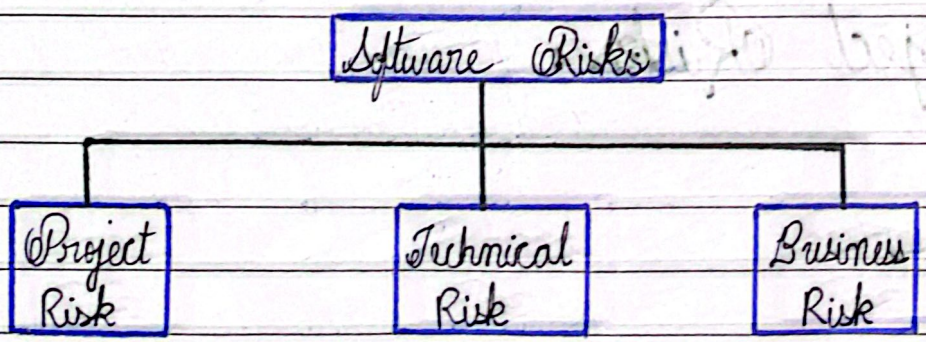
- Use of new unfamiliar technology.
- complex system architecture.
- integration difficulties between system components.

Business Risks: Business risks affect the organization's ability to benefit from the system.

Examples:

- System not meeting market needs
- loss of customer trust
- project cancellation due to financial reasons.

- **Classification of Software Risks:-**



Date

3) Risk Identification :-

The first step in risk analysis is identifying potential risks. Common techniques for identifying risks include :

- brainstorming sessions
- expert consultation
- reviewing previous project experiences.
- analyzing project documentation

Once risks are identified, they are recorded in a risk register.

4) Risk Assessment :-

After identifying risks, engineers must evaluate :

- the probability that the risk will occur.
- the potential impact if the risk occurs.

Risks with high probability and high impact require immediate attention.

Scanned with CamScanner

Date

5) Risk Mitigation :-

Risk mitigation refers to strategies used to reduce or eliminate risks. Examples of risk mitigation strategies include:

- using proven technologies instead of experimental ones.
- conducting regular project reviews
- improving communication among stakeholders.
- creating backup plans for critical tasks.

6) Impact Analysis :-

Impact analysis is the process of analyzing the consequences of a requirement change.
When a requirement changes, it may affect several components of the system such as:

- system design
- database structure
- program code
- testing procedure

Impact analysis helps engineers understand the effects of these changes before implementing them.

Scanned with CamScanner

Date

7) Importance of Impact Analysis :-

Impact analysis is important because it :

- prevents unexpected system failures.
- helps estimate development cost and time.
- ensure that system changes are properly managed.
- reduces development risks.

Without impact analysis, requirement changes can lead to system inconsistencies and project delays.

8) Summary :-

Risk analysis helps identify and evaluate potential problems that may affect the success of a software project. By understanding these risks early, teams can implement strategies to minimize their impact.

Impact analysis focuses on evaluating how requirement changes affect different parts of the system. This process helps developers manage system changes efficiently and maintain system stability.

Scanned with CamScanner

Week #06

Date

1) Introduction :-

In software development, requirements and system architecture are closely related. Requirements describe what the system should do, while architecture describes how the system will be structured to fulfill those requirements.

The architecture of a system defines the high-level structure, components, and communication between different parts of the system.

A well-designed architecture ensures that the system satisfies both functional and non-functional requirements.

2) What is Software Architecture :-

Software architecture refers to the overall structure of a software system. It defines :

- major system components.
- relationships between components.
- communication mechanisms.
- system organization.

Architecture acts as a blueprint of the system.

Scanned with CamScanner

Date

3) Interaction Between Requirements And Architecture :-

Requirements influence the design of the architecture, while architecture may also influence how requirements are implemented.

For example:

- performance requirements affect system processing design.
- security requirements influence authentication mechanisms.
- scalability requirements affect deployment structure.

Thus, requirements and architecture must be designed together.

Date

4) Importance of Aligning Requirements & Architecture :-

The alignment between requirements and architecture is important because it :

- ensures system design satisfies user needs.
- improves system performance and reliability.
- reduces development errors.
- simplifies future system maintenance.

If architecture does not support requirements, the system may fail to meet stakeholder expectations.

5) Requirements Elicitation :-

Requirements elicitation is the process of collecting system requirements from stake-holders and other sources.

It is one of the most important stages of requirements engineering because incorrect or incomplete requirements can lead to system failure. The main goal of elicitation is to understand the needs, expectations, and constraints of stakeholders.

Scanned with CamScanner

Date

6) Stakeholders :-

Stakeholders are individuals or groups who are directly or indirectly affected by the system. Example include :

- customers
- system users
- developers
- project managers
- business managers
- system administrators

Each stakeholder may have different expectations from the system.

7) Elicitation Process :-

The requirement elicitation process generally includes the following steps:

1. Identify stakeholders
2. Gather information about system needs.
3. Analyze the collected information.
4. Resolve conflicts between requirements.
5. Document the requirements.

Scanned with CamScanner

Date

8) Challenges in Requirements Elicitation :-

Requirements elicitation may face several challenges:

Communication problems
Stakeholders may not clearly explain their needs.

Conflicting requirements
Different stakeholders may have different expectations.

Changing requirements
Stakeholders needs may change during development.

Incomplete knowledge
Sometimes users do not fully understand their own requirements.

9) Summary :-

Requirements and architecture are strongly interconnected. Requirements define what the system must achieve, while architecture provides the structural design to implement those requirements. Requirements elicitation is the process of gathering system requirements from stakeholders and other sources. It plays a critical role in ensuring the success of a software project.

Scanned with CamScanner

Week #07

Date _____

1) Sources of Requirements :-

Requirements may originate from several sources within an organization. Common sources include:

System Users: Users interact directly with the system and provide valuable information about required features.

Customers: Customers specify business goals and expectations.

Business Processes: Existing business processes often determine system requirements.

Existing Systems: Older systems may provide insight into required functionality.

2) Requirements Elicitation Techniques:

Software engineers use various techniques to gather requirements from stakeholders.

Interviews :-

Interviews involve direct communication between developers and stakeholders.

Types of interviews include:

Scanned with CamScanner

Date

- structured interviews
- semi - structured interviews.
- unstructured interviews.

Advantages:

- detailed information
- direct clarification of requirements.

Questionnaires :-

Questionnaires are useful when information must be collected from a large number of stakeholders.

Advantages :-

- quick data collection
- easy to distribute.

Observation :-

Observation involves watching users perform their tasks in real environments.

Advantages :-

- identifies hidden requirements
- improves understanding of user behavior.

Scanned with CamScanner

Date

Workshops :-

Workshops involve group discussions between stakeholders and developers.

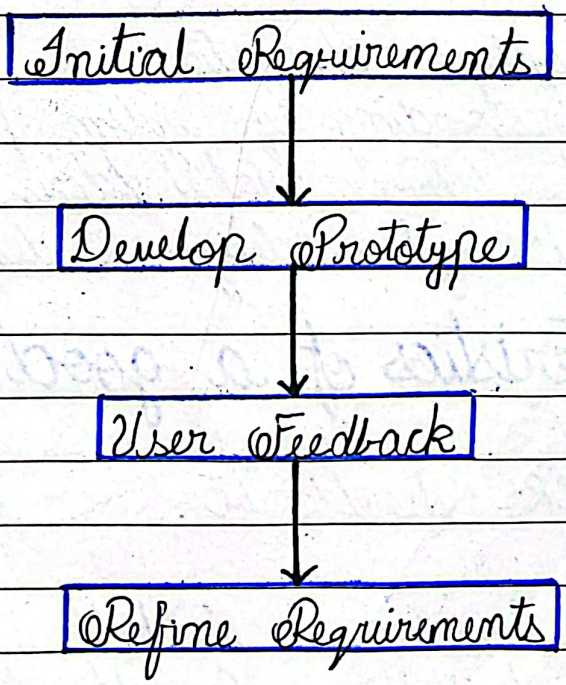
Benefits:

- encourages collaboration
- resolves requirement conflicts.

Prototyping :-

Prototyping involves building a preliminary model of the system. Stakeholders can interact with the prototype and provide feedback.

Prototyping Process :-



Scanned with CamScanner

Date

3) Requirement Specification :-

Requirement specification is the process of documenting system requirements clearly and formally.

The document that contains all system requirements is called the Software Requirements Specification (SRS).

The SRS document serves as a reference for developers, testers, and project managers.

4) Purpose of SRS :-

The Software Requirements Specification provides:

- detailed system description.
- clear understanding of system functionality.
- agreement between stakeholders & developers.
- guidance for system design & development.

5) Characteristics of a good SRS :-

A good SRS should be :

Correct	Consistent	Modifiable	Unambiguous
Complete	Verifiable	Traceable	

Date

6) Documentation of Requirements :-

Requirements documentation helps ensure that all stakeholders understand the system clearly. Documentation usually includes :

- system overview
- function requirements
- non-functional requirement
- constraints
- system models and diagrams.

7) Summary :-

Requirements may originate from multiple sources including users, business processes, and regulatory constraints.

Different elicitation techniques such as interviews, observation, and prototyping help developers gather accurate requirements.

Requirement specification is the process of documenting system requirements in the Software Requirements Specification (SRS) document.

Scanned with CamScanner

Week # 09

Date

1) Specification Techniques :-

Requirement specification techniques are used to represent requirements clearly and systematically so that all stakeholders can understand them. Different techniques are used depending on system complexity.

Natural Language Specification :-

This is the simplest technique where requirements are written in plain english.

Example:

"The system shall allow users to log in using username and password".

Advantages :-

- easy to understand
- no technical knowledge required.

Disadvantages :-

- may cause ambiguity.
- difficult to verify

Scanned with CamScanner

Date

Structured Specification :-

In this technique, requirements are written ~~as~~ in a structured format using templates.

Example Format :-

- Requirement ID
- Description
- Inputs
- Outputs
- Constraints

This reduces ambiguity and improves clarity.

Graphical Models :-

Graphical representations help visualize system behavior.

Examples include :

- Data Flow Diagrams (DFD)
- Use Case Diagrams
- UML Diagrams

Scanned with CamScanner

Date

Mathematical Specification :-

This technique uses mathematical expressions to define system behavior.

Advantages:

- precise and unambiguous

Disadvantages:

- difficult to understand for non-technical stakeholders.

Requirements Validation :-

Requirements validation is the process of ensuring that:

- requirements are correct
- requirements are complete
- requirements meet user needs

It answers the most important question:

"Are we building the right system".

Date

Importance of Requirements Validation :-

Validation is critical because :

- errors are cheaper to fix early.
- prevents system failure.
- ensures customer satisfaction.
- improves overall software quality.

Requirements Validation Techniques :-

1. Requirements Reviews :-

Stakeholders and developers read and analyze the requirements document to identify :

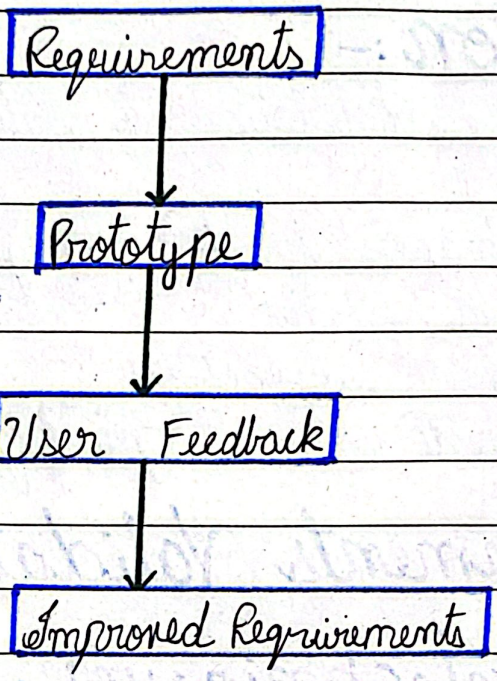
- errors
- missing requirements
- inconsistencies

2. Prototyping :-

A prototype (early model of system) is developed and shown to users.
Users give feedback, which helps refine requirements.

Date

Prototyping Flow :-



3. Test Case Generation :-

Test cases are created based on requirements to check if they are testable. If a requirement cannot be tested → it is likely unclear.

4. Automated Validation Tools :-

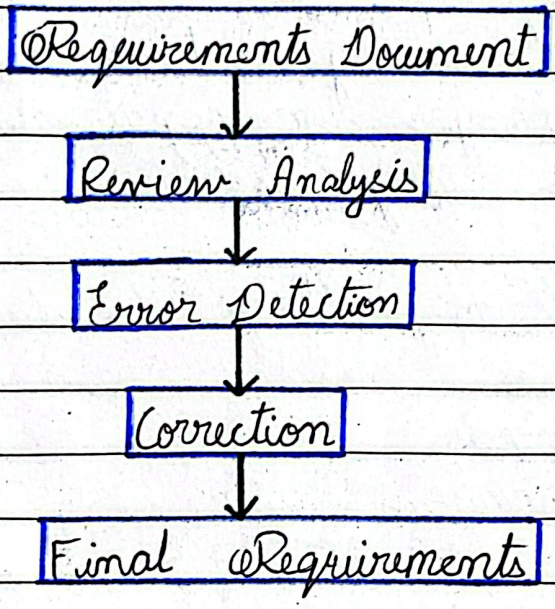
Software tools are used to ::

- check consistency
- detect missing requirements
- validate structure

Scanned with CamScanner

Date

Validation Process :-



Summary :-

Specification techniques help represent requirements clearly using natural language, structured formats, diagrams, and mathematical models.

Requirements validation ensures that the system is built correctly by checking requirements through reviews, prototyping, and testing techniques.

Scanned with CamScanner

Week #10

Date _____

1) Requirements Management :-

Requirements management is the process of handling changes in requirements throughout the software lifecycle.

Requirements may changes due to :

- new user needs
- business changes
- technical constraints

Managing these changes properly is essential for project success.

2) Activities in Requirements Management :-

Key activities include :

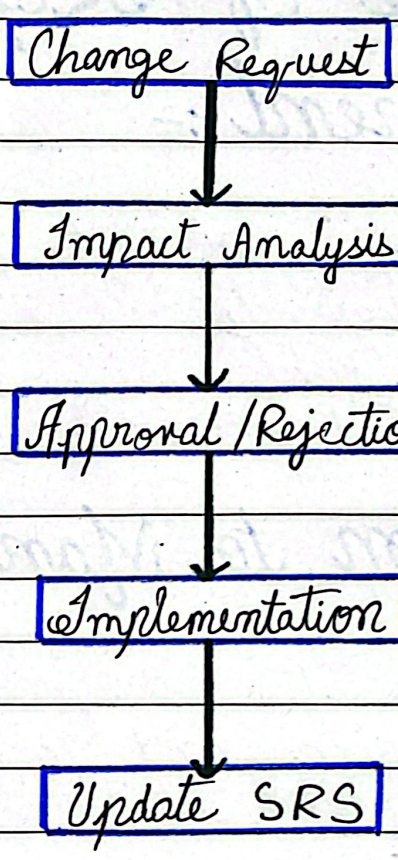
- change management
- version control
- requirement tracking
- impact analysis

3) Change Management Process :-

When a requirement changes, it must go through a proper process.

Scanned with CamScanner

Date



4) Requirement Traceability :-

Traceability is the ability to track requirements throughout the development process.

It helps answer :

- where a requirement is used
- how it is implemented.
- what happens if it changes

Date

5) Importance Of Requirements Management :-

- handles changing requirements effectively.
- improves system quality
- reduces project risks
- ensures project stays on track.

6) Introduction to Management :-

Management in software engineering refers to planning, organizing, and controlling software development activities.

It includes:

- project planning
- resource management
- risk management
- quality control

7) Summary :-

Requirements management ensures that all requirement changes are properly handled throughout the software lifecycle. Effective management improves system quality and reduces project risks.

Scanned with CamScanner

Week # 11

Date

1) Requirements Management Problems :-

Managing requirements is challenging due to several problems.

i) Changing Requirements :-

Requirements may frequently change, causing instability.

ii) Conflicting Requirements :-

Different stakeholders may have different expectations.

iii) Lack of Communication :-

Poor communication leads to misunderstandings.

iv) Incomplete Requirements :-

Some requirements may be missing or unclear.

2) Managing These Problems :-

Solution include :

- proper documentation
- regular stakeholder communication
- use of management tools.
- change control processes.

Scanned with CamScanner

Date

3) Acquisition Organization :-

An acquisition organization is one that purchases or outsources software development. Instead of developing software internally, they hire vendors.

4) Challenges in Acquisition Organization :-

i) **Communication Gap:** Client and supplier may misunderstand requirements.

ii) **Contract Issues:** Requirements must be clearly defined in contract.

iii) **Requirement Changes:** Changes are difficult after contract agreement.

iv) **Dependency on Supplier:** Client depends on external team.

5) Managing Requirements in Acquisition Organizations :-

Scanned with CamScanner

Date

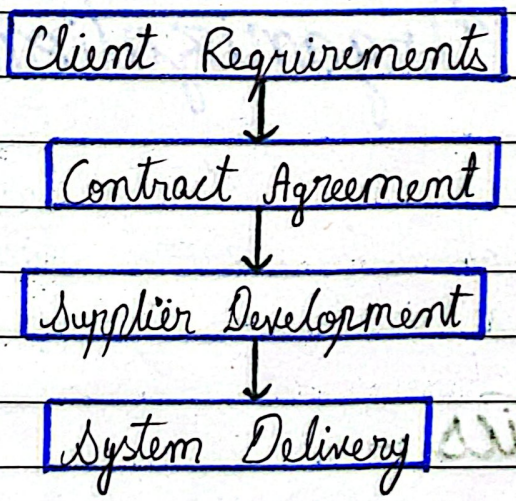
Clear Contract Specification: All requirements must be well-defined.

Regular Communication: Frequent meetings with supplier.

Change Control System: Formal approval required for changes.

Monitoring Progress: Track development and requirement implementation.

Process Diagram :-



Summary :-

Requirements management problems include changing requirements, conflicts, communication issues, and scope creep.
In acquisition organizations, managing requirements becomes more challenging due to external suppliers, contracts and communication gaps.

Scanned with CamScanner

Week # 12

Date _____

1) Introduction :-

In software engineering, organizations develop systems in different business models. The two major types are :

- Supplier Organizations
- Product Organizations

Understanding the difference is important because :

It affects how requirements are gathered, managed, and changed.

2) Supplier Organizations :-

Supplier organizations develop software for external clients. They work based on contracts and customer requirements.

Characteristics :-

- work for multiple clients
- follow strict contracts
- focus on delivery deadlines.

3) Challenges for Supplier Organizations :-

Scanned with CamScanner

Date

- unclear client requirements
- communication issues
- changing requirements

4) Product Organizations :-

Product organizations develop software products for the market instead of specific clients.

Examples:

- mobile apps
- SaaS platforms
- commercial software

Characteristics :-

- focus on market needs
- continuous product improvement
- long-term development.

Supplier vs Product Organization :-

Supplier Organization	Product Organization
Change is difficult	Change is flexible
Client-based	Market-based
Contract driven	Product vision-driven
Fixed requirements	Evolving requirements
Short-term projects	Long-term projects/development

Scanned with CamScanner

Week #13

Date _____

Requirements Engineering for Agile Methods :-

1) Introduction :-

Agile methods focus on flexibility and continuous change in software development. Unlike traditional methods, requirements are not fixed at the beginning.

2) Agile Requirements :-

In agile development.

- requirements are written as user stories
- requirements evolve over time
- continuous feedback is taken from users.

Example User Story :-

"As a user, I want to log in so that I can access my account".

3) Characteristics of Agile Requirements :-

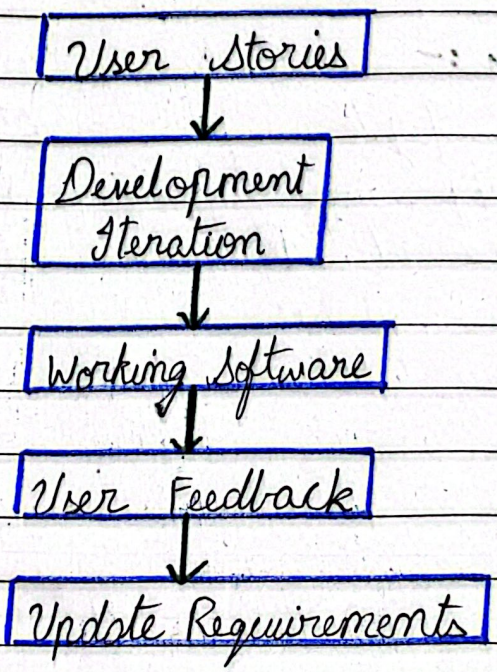
- simple and short

Scanned with CamScanner

Date

- flexible
- continuously updated
- focused on user needs

4) Agile Requirements Process :-



5) Advantages of Agile Requirements :-

- handles changing requirements easily.
- improves customer satisfaction.
- faster delivery of features.
- reduces risk

Date

6) Challenges :-

- lack of detailed documentation.
- difficult to manage large systems.
- require strong communication.

7) Summary :-

Agile requirements engineering focuses on flexibility, continuous feedback, and iterative development. Requirements are represented as user stories and evolve throughout the project.

Scanned with CamScanner